```
#include <iostream>
#include <climits>
using namespace std;

// Function to print optimal parenthesization
void printOptimalParenthesis(int i, int j, int n, int split[][100]) {
    if (i == j) {
        cout << "A" << i;
        return;
    }

    cout << "(";
    printOptimalParenthesis(i, split[i][j], n, split);
    printOptimalParenthesis(split[i][j] + 1, j, n, split);
    cout << ")";
}

int main() {
    int n;

    cout << "Enter number of matrices: ";
    cin >> n;

    int p[100];
    int dp[100][100];
    int split[100][100];

    cout << "Enter " << n + 1 << " dimensions: ";
    for (int i = 0; i <= n; i++) {
        cin >> p[i];
    }
```

```
// Cost of multiplying one matrix
for (int i = 1; i <= n; i++) {
   dp[i][i] = 0;
}

// Chain length
for (int length = 2; length <= n; length++) {

   for (int i = 1; i <= n - length + 1; i++) {

      int j = i + length - 1;
      dp[i][j] = INT_MAX;

      // Try every possible split
      for (int k = i; k < j; k++) {

         int cost = dp[i][k]
               + dp[k + 1][j]
               + p[i - 1] * p[k] * p[j];

         if (cost < dp[i][j]) {
            dp[i][j] = cost;
            split[i][j] = k;
         }
      }
   }
}

cout << "\nMinimum number of scalar multiplications: "
    << dp[1][n] << endl;
```

```
    cout << "Optimal Parenthesization: ";
    printOptimalParenthesis(1, n, n, split);
    cout << endl;

    return 0;
}
```

```
Enter number of matrices: 3
Enter 4 dimensions: 10 20 30 40

Minimum number of scalar multiplications: 18000
Optimal Parenthesization: ((A1A2)A3)

=== Code Execution Successful ===
```